Adaptive Heuristic Search and Game Playing

A Mini-Textbook for Six Two-Hour Lectures

Artificial Intelligence Course

2026 edition

Preface

This text develops heuristic and adversarial search as one continuous subject. The common problem is that a decision maker cannot expose the entire future. It must construct a useful portion of a search tree, estimate what lies beyond the current frontier, and decide where computation is worth spending.

In single-agent search, the estimate is a heuristic $h(n)$. In two-player zero-sum games, it is an evaluation $E(s)$ at a depth-limited frontier. An adaptive search method improves either the estimate or the way it allocates search effort by using experience. This viewpoint connects A*, Adaptive A*, LRTA*, bounded real-time search, MinMax, alpha–beta pruning, transposition tables, move ordering, quiescence search, and learned evaluation functions.

The six chapters correspond to six lectures of two hours each. Every chapter contains definitions, algorithms, examples, theoretical results, implementation notes, and exercises. The appendices contain solution sketches, a programming project, a glossary, and references. The notation assumes a first course in algorithms, but no prior knowledge beyond elementary graph search.

The unifying loop

The entire module can be summarized by the following controlled-search loop:

$$\boxed{\text{construct a partial future}} \longrightarrow \boxed{\text{evaluate its frontier}} \longrightarrow \boxed{\text{choose}} \longrightarrow \boxed{\text{learn and repeat}}.$$

Suggested use

For each lecture, read the chapter introduction and definitions before class. During class, work through the main algorithm and at least one trace by hand. After class, answer the conceptual questions before attempting the programming exercises. The code-like presentations are pseudocode: implementation details such as priority-queue updates, hashing, and tie-breaking must be made explicit in an actual program.

Contents

Preface **ii**

1 Heuristics as Search Control **1**

1.1 Search problems and their graphs . . . 1
1.2 Best-first search . . . 1
1.3 Heuristics . . . 2
1.4 Why A* is optimal . . . 3
1.5 Heuristic dominance and geometry . . . 3
1.6 Implementation details that affect results . . . 4
1.7 Worked comparison . . . 4
1.8 Exercises . . . 4

2 Adaptive Heuristic Search **5**

2.1 What makes search adaptive? . . . 5
2.2 Bellman equations and local backups . . . 5
2.3 Adaptive A* . . . 6
2.4 Online search and LRTA* . . . 6
2.5 A hand trace . . . 7
2.6 Heuristic depressions . . . 7
2.7 Convergence and costs of learning . . . 7
2.8 Exercises . . . 8

3 Search under Resource Constraints **9**

3.1 When optimal A* is not the right algorithm . . . 9
3.2 Weighted A* . . . 9
3.3 Anytime search . . . 10
3.4 Memory-bounded methods . . . 10
3.5 Bounded lookahead . . . 10
3.6 Dynamic environments . . . 11
3.7 From passive worlds to opponents . . . 11

3.8 Exercises . . . 11

4 Adversarial Search and MinMax **13**

4.1 A formal game model . . . 13
4.2 Strategies, not paths . . . 13
4.3 Deriving MinMax . . . 14
4.4 Worked tree . . . 14
4.5 Correctness by backward induction . . . 14
4.6 Complexity . . . 15
4.7 The relation to shortest-path search . . . 15
4.8 Exercises . . . 15

5 Making MinMax Practical **16**

5.1 Depth-limited search . . . 16
5.2 The horizon effect . . . 16
5.3 Alpha–beta bounds . . . 17
5.4 Why pruning is safe . . . 17
5.5 Move ordering . . . 18
5.6 Iterative deepening . . . 18
5.7 Transposition tables . . . 18
5.8 Negamax as an implementation simplification . . . 19
5.9 Exercises . . . 19

6 Adaptive Game-Playing Agents **20**

6.1 Where adaptation enters a game engine . . . 20
6.2 Quiescence search . . . 20
6.3 Adaptive move ordering . . . 21
6.4 Learning an evaluation function . . . 21
6.5 Search as a teacher . . . 21
6.6 A modern search agent . . . 22
6.7 Experimental methodology . . . 22
6.8 The completed conceptual arc . . . 23
6.9 Exercises . . . 23

A Solution Sketches **24**

B Programming Project: A Search Agent in Two Worlds **26**

B.1 Part A: Adaptive navigation . . . 26
B.2 Part B: Reduced Connect Four . . . 26

B.3 Suggested assessment rubric 27

C Notation and Glossary **28**

D Further Reading **29**

Closing Perspective **31**

Chapter 1

Heuristics as Search Control

Learning goals

- Formalize a shortest-path search problem and distinguish tree from graph search.
- Explain the roles of $g(n)$, $h(n)$, and $f(n)$ in UCS, Greedy, and A*.
- Distinguish admissibility from consistency and use both properties correctly.
- Predict how heuristic quality and tie-breaking change the explored region.
- Implement A* with duplicate detection and path improvement.

1.1 Search problems and their graphs

A deterministic search problem is a tuple

$$\mathcal{P} = (S, A, T, c, s_0, G),$$

where S is a set of states, $A(s)$ is the set of actions available in state s, $T(s, a)$ is the successor state, $c(s, a) > 0$ is the step cost, s_0 is the initial state, and $G \subseteq S$ is the set of goal states. A path $\pi = (s_0, a_0, s_1, \ldots, s_k)$ has cost

$$C(\pi) = \sum_{i=0}^{k-1} c(s_i, a_i).$$

The task is to return a path from s_0 to some $g \in G$, normally one of minimum cost.

A *state* belongs to the problem. A *search node* belongs to the algorithm: it records a state, a parent pointer, an incoming action, a path cost, and often a depth. The same state may be represented by many tree nodes. Graph search detects these duplicates and keeps the best known path to each state.

Key idea. The search tree is not the problem itself. It is a computational object built by the algorithm. Search control determines which part of that object becomes explicit.

1.2 Best-first search

Best-first search maintains a frontier OPEN ordered by a numerical key $f(n)$. At every iteration it removes a node of minimum key, tests or expands it, and inserts useful successors.

Listing 1.1: Generic best-first graph search

```
function BEST-FIRST(start, goals, f):
    OPEN   <- priority queue containing start
    g[start] <- 0
    parent[start] <- null

    while OPEN is not empty:
        n <- remove a node with minimum f-value from OPEN
        if n is a goal:
            return path reconstructed through parent
        if n has already been expanded with a no-larger g-value:
            continue
        record n as expanded with value g[n]

        for each successor n' of n with step cost c(n,n'):
            tentative <- g[n] + c(n,n')
            if n' is new or tentative < g[n']:
                g[n'] <- tentative
                parent[n'] <- n
                insert or update n' in OPEN using f(n')
    return failure
```

Three important choices are:

$$\begin{aligned} \text{Uniform-Cost Search (UCS):} &\qquad f(n) = g(n), \\ \text{Greedy Best-First Search:} &\qquad f(n) = h(n), \\ \text{A*:} &\qquad f(n) = g(n) + h(n). \end{aligned}$$

Here $g(n)$ is the cost already paid and $h(n)$ estimates the remaining cost. UCS ignores destination information. Greedy search ignores the cost already paid. A* balances the two.

1.3 Heuristics

Let $h^*(s)$ be the exact cost of a cheapest path from s to a goal, with $h^*(s) = \infty$ if no goal is reachable.

Definition 1.1 (Admissibility)**.** A heuristic is admissible if

$$0 \le h(s) \le h^*(s)$$

for every state s, and $h(g) = 0$ for every goal g.

An admissible heuristic is optimistic: it never claims that the remaining problem is more expensive than it really is.

Definition 1.2 (Consistency)**.** A heuristic is consistent (or monotone) if, for every transition from s to s',

$$h(s) \le c(s, s') + h(s'),$$

and $h(g) = 0$ at goals.

Consistency is a triangle inequality. It implies that f is nondecreasing along every path:

$$g(s') + h(s') \ge g(s) + h(s).$$

Consistency implies admissibility when a goal is reachable. The converse need not hold on a graph.

Example 1.3 (Grid heuristics)**.** On a four-neighbour grid with unit step cost and goal (x_g, y_g), Manhattan distance

$$h_M(x, y) = |x - x_g| + |y - y_g|$$

is admissible and consistent. If diagonal moves of cost 1 are allowed, it is not admissible; Chebyshev distance

$$h_C(x, y) = \max\{|x - x_g|, |y - y_g|\}$$

is appropriate. Euclidean distance is admissible whenever a move costs at least its Euclidean length.

1.4 Why A* is optimal

Let C^* be the optimal solution cost. For every node n on an optimal path, admissibility gives

$$f(n) = g(n) + h(n) \leq g(n) + h^*(n) = C^*.$$

Any suboptimal goal g' has $f(g') = g(g') > C^*$. Therefore A* cannot select a suboptimal goal while an optimal-path frontier node remains available with $f \leq C^*$.

Theorem 1.4 (A* tree-search optimality)**.** *With positive step costs and an admissible heuristic, A* tree search returns an optimal solution, provided a solution is found and nodes are goal-tested when removed from* OPEN*.*

For graph search, consistency makes the first expansion of a state definitive. If only admissibility is known, a better path to an already expanded state may be discovered; correctness then requires reopening it.

Theorem 1.5 (A* graph-search optimality)**.** *With positive step costs and a consistent heuristic, A* graph search with a best-g table returns an optimal solution and need not reopen expanded states.*

1.5 Heuristic dominance and geometry

If h_1 and h_2 are admissible and

$$h_2(s) \geq h_1(s) \quad \text{for every } s,$$

then h_2 *dominates* h_1. Apart from tie effects, A* with h_2 expands no more nodes than A* with h_1. The limiting cases are revealing:

$$h(s) = 0 \Rightarrow \text{UCS}, \qquad h(s) = h^*(s) \Rightarrow \text{only optimal directions need be explored.}$$

A heuristic also defines an implicit geometry. The set

$$\{s : g(s) + h(s) \leq C\}$$

is the region A* can consider before accepting a solution of cost C. With $h = 0$, this resembles a cost ball around the start. An informative heuristic stretches it toward the goal and suppresses irrelevant directions.

1.6 Implementation details that affect results

- **Goal test.** In A*, accept a goal when it is popped, not when first generated.
- **Priority updates.** A newly improved g must change the queue key. A common alternative is to insert a new entry and discard stale entries later.
- **Tie-breaking.** Among equal f, larger g often reduces expansions by preferring deeper nodes, without changing optimality.
- **Zero-cost edges.** They complicate termination and can create large equal-f plateaus. State assumptions explicitly.
- **Memory.** A* can store exponentially many frontier and explored nodes; excellent time performance does not make it memory-bounded.

1.7 Worked comparison

Suppose a graph has edges

$$S \xrightarrow{2} A, quad S \xrightarrow{1} B, quad A \xrightarrow{5} G, quad B \xrightarrow{2} C, quad C \xrightarrow{2} G,$$

and heuristic values $h(S) = 4, h(A) = 1, h(B) = 3, h(C) = 1, h(G) = 0$. Greedy selects A after S and may reach a cost-7 goal first. A* has $f(A) = 3$ and $f(B) = 4$, so it first expands A; it generates G with $f = 7$, but then expands B and C, producing a cost-5 goal. The crucial rule is that a generated goal is not automatically accepted.

1.8 Exercises

1. Give an admissible but inconsistent heuristic on a graph of four states. Show where A* must reopen a state if it uses graph search.
2. Prove that the maximum of two admissible heuristics is admissible. Is their sum necessarily admissible?
3. Trace UCS, Greedy, and A* on the worked graph. Record Open after every expansion and state the returned path.
4. Design admissible heuristics for the eight-puzzle. Compare misplaced tiles and Manhattan distance by dominance.
5. Explain precisely why accepting a goal on generation can make A* suboptimal.
6. Implement the grid experiment using $h = 0$, Manhattan distance, and 1.5 times Manhattan distance. Report path cost, expansions, and peak frontier size.

Chapter 2

Adaptive Heuristic Search

Learning goals

- Distinguish a static heuristic from a heuristic updated by search experience.
- Derive Bellman-style heuristic backups.
- Explain Adaptive A* and when its updates remain admissible.
- Trace LRTA* and explain heuristic depressions and convergence.
- Separate learning a search representation from memorizing a solution path.

2.1 What makes search adaptive?

Ordinary A* treats h as fixed. Adaptive search produces a sequence

$$h_0(s), h_1(s), h_2(s), \dots$$

from information obtained during planning or execution. The update can improve future searches between episodes, as in Adaptive A*, or change behaviour inside one online episode, as in LRTA*.

Adaptation is useful when problems share structure: repeated navigation to a fixed goal, replanning after local map changes, repeated game positions, or many queries on the same state space. Search effort from one episode becomes information rather than being discarded.

2.2 Bellman equations and local backups

For deterministic shortest paths, the perfect cost-to-go satisfies

$$h^*(s) = \begin{cases} 0, & s \in G, \\ \min_{a \in A(s)} \{c(s,a) + h^*(T(s,a))\}, & s \notin G. \end{cases}$$

This suggests the one-step backup

$$h(s) \leftarrow \min_{a \in A(s)} \{c(s,a) + h(T(s,a))\}.$$

If the current estimate is too low, the backup raises it toward local consistency. An update should generally be monotone in learning algorithms:

$$h(s) \leftarrow \max \left\{ h(s), \min_a [c(s,a) + h(T(s,a))] \right\}.$$

The maximum prevents new experience from making the estimate less informed.

2.3 Adaptive A*

Adaptive A* addresses a sequence of searches with a common goal. During an A* episode, let $g(s)$ be the start-to-state cost, let g^* be the cost of the solution found, and let CLOSED be the set of expanded states. Update

$$h_{\text{new}}(s) = g^* - g(s), \qquad s \in \text{CLOSED}.$$

Leave the remaining heuristic values unchanged.

Listing 2.1: Adaptive A* over repeated episodes

```
procedure ADAPTIVE-A-STAR(start, goal, h):
    path, g, CLOSED <- A-STAR(start, goal, h)
    if path exists:
        solutionCost <- g[goal]
        for each s in CLOSED:
            h[s] <- max(h[s], solutionCost - g[s])
    return path, h
```

Why is $g^* - g(s)$ meaningful? When A* terminates, every expanded state s satisfies

$$g(s) + h(s) \leq g^*.$$

Thus the update raises its f-value to the cost of the just-discovered solution. For repeated searches under the usual fixed-goal assumptions, the new value remains a lower bound on the true remaining distance.

Proposition 2.1 (Informal Adaptive A* guarantee)**.** *Assume deterministic costs, a fixed goal, an initially consistent heuristic, and cost changes that do not invalidate the lower-bound interpretation. Then Adaptive A* preserves admissibility, never decreases heuristic values, and can reduce expansions on later related searches.*

The qualification about cost changes matters. If an edge becomes cheaper, an old lower bound can become an overestimate. If obstacles are only added, true distances do not decrease, so earlier lower bounds remain safe.

2.4 Online search and LRTA*

In offline planning, the agent computes a complete route and then executes it. In online search, computation and action alternate. The agent may initially know only local successors and must act before it can solve the whole problem.

Learning Real-Time A* (LRTA*) performs shallow lookahead, updates the present state, and moves to the most promising successor.

Listing 2.2: One-step LRTA* for a deterministic environment

```
procedure LRTA-STEP(current, h):
    observe successors Succ(current) and their step costs
    best <- min over s' in Succ(current) of c(current,s') + h[s']
    h[current] <- max(h[current], best)
```

```
    next <- a successor attaining best
    execute the action leading to next
    return next
```

Repeated trials typically improve h and reduce wandering. The agent learns a cost landscape rather than a literal instruction list. This matters when the start changes or multiple routes share states.

2.5 A hand trace

Consider states A, B, C, G with transitions $A \leftrightarrow B$ of cost 1, $B \leftrightarrow C$ of cost 1, and $C \leftrightarrow G$ of cost 1. Suppose the initial heuristic is 0 everywhere and ties are resolved toward the previous state when possible.

At A, the backup gives $h(A) = 1$ and the agent moves to B. At B, both A and C appear to cost 1 ahead if $h(C) = 0$ but A now costs 2; therefore the agent moves to C. At C, it moves to G. In a graph with dead ends or broad zero-valued regions, repeated visits may be required before the learned gradient points consistently outward.

2.6 Heuristic depressions

A heuristic depression is a connected region whose interior heuristic values are low relative to the cost of leaving. A greedy local method is attracted to the low estimates and must raise many states before escaping. Obstacles in a Manhattan grid create a standard example: states behind a U-shaped wall look geometrically close to the goal although every real route must first move away.

Possible responses include:

- deeper lookahead;
- updating more than the current state;
- weighting estimated cost differently;
- marking or avoiding recently revisited regions;
- learning from previous trials.

Each exchanges computation, memory, solution quality, and reaction time.

2.7 Convergence and costs of learning

In a finite deterministic state space with positive costs, a reachable goal, fair tie-breaking, and an admissible initialization, LRTA*-style methods have useful convergence guarantees: heuristic values rise only finitely often on a finite cost lattice, and repeated trials eventually follow an optimal route under standard assumptions. The first trial, however, can be very expensive.

Three different quantities must not be confused:

1. **planning work**: node expansions or backups;

2. **execution cost**: distance actually travelled by the online agent;
3. **solution quality**: cost of the final or converged route.

An online algorithm can use little computation per move but travel a long path.

Key idea. Adaptive search does not merely learn a solution. It improves a representation of what remains costly, making future allocation of search effort more useful.

2.8 Exercises

1. Derive the Adaptive A* update from the inequality used at A* termination.
2. Construct a map change that makes a previously admissible learned heuristic inadmissible. What restriction on changes avoids the problem?
3. Trace three LRTA* trials through a five-state graph with a dead end. Record the heuristic after every action.
4. Explain why a one-step backup resembles dynamic programming but does not by itself solve the whole Bellman system.
5. Design an experiment that distinguishes learning a reusable heuristic from memorizing the previous path.
6. On a grid with a U-shaped obstacle, compare A*, repeated A*, Adaptive A*, and LRTA* using expansions, travel cost, and stored states.

Chapter 3

Search under Resource Constraints

Learning goals

- Analyze deliberate trade-offs between solution cost, time, and memory.
- State the suboptimality guarantee of Weighted A* under clear assumptions.
- Explain anytime search, bounded lookahead, and RTAA*.
- Recognize why real-time constraints change the optimization objective.
- Use bounded search to motivate adversarial game trees.

3.1 When optimal A* is not the right algorithm

A* answers an idealized question: find an optimal path given enough time and memory. A deployed agent may instead have a deadline, a strict memory budget, or a changing environment. It may need a usable answer immediately and improve it if more time remains. Search then becomes a multi-objective problem involving solution cost, computation time, memory, and action latency.

3.2 Weighted A*

Weighted A* uses

$$f_w(n) = g(n) + wh(n), \qquad w \geq 1.$$

Increasing w makes the search more goal-directed. At $w = 1$ it is A*; as w becomes large it approaches Greedy search.

Theorem 3.1 (Weighted A* bound)**.** *For an admissible heuristic and the standard termination and reopening rules, Weighted A* returns a solution of cost*

$$C \leq wC^*.$$

Proof sketch. Before termination there is a frontier node n on an optimal path. For it, $g(n) + h^*(n) = C^*$ and $h(n) \leq h^*(n)$. Therefore

$$f_w(n) = g(n) + wh(n) \leq g(n) + wh^*(n) \leq wC^*.$$

The returned goal has $h = 0$, so its key equals its path cost and cannot exceed the minimum frontier key at selection. □

The bound is worst-case. In practice, moderate weights can reduce expansions substantially while increasing path cost very little. The weight is a control parameter, not a universal constant.

3.3 Anytime search

An anytime algorithm quickly returns a feasible solution and then improves it. The current solution cost C_{inc} is an upper bound on C^*. A frontier lower bound, such as $L = \min_{n \in \text{OPEN}} g(n) + h(n)$, provides a certificate:

$$L \leq C^* \leq C_{inc}, \qquad \frac{C_{inc}}{L}$$

is a multiplicative quality bound when $L > 0$.

A simple design runs Weighted A* with a large w, stores the incumbent, then decreases w while reusing search information. More sophisticated methods repair keys and inconsistency sets rather than restarting. The defining feature is observable improvement plus a meaningful stopping state at every deadline.

3.4 Memory-bounded methods

Iterative Deepening A* (IDA*) performs depth-first searches under successive f-thresholds. It uses linear memory in the current depth but may regenerate many nodes. Recursive Best-First Search stores a best alternative value while recursing and also trades memory for repeated work. Simplified Memory-Bounded A* drops the least promising leaves when memory is full and backs their values up to parents.

There is no free reduction in memory: forgotten structure may have to be reconstructed. The correct method depends on whether memory, time, or action latency is the dominant constraint.

3.5 Bounded lookahead

A real-time agent can search only a local envelope around its current state. Let L be the set of expanded states and F the frontier produced before the lookahead limit. A generic cycle is:

$$\text{look ahead} \to \text{evaluate } F \to \text{update } L \to \text{act} \to \text{repeat}.$$

RTAA* (Real-Time Adaptive A*) performs a bounded A* search and uses the best frontier value to update expanded states. Let

$$f_{\min} = \min_{u \in F} \{g(u) + h(u)\}.$$

For each expanded state s, a common update is

$$h(s) \leftarrow f_{\min} - g(s).$$

The agent then moves along the partial path toward the best frontier state, possibly for one action or for several actions before replanning.

Listing 3.1: Schematic RTAA* episode

```
procedure RTAA-STEP(current, goal, h, expansionLimit):
    CLOSED, OPEN, g, parent <- bounded A* from current
    frontierBest <- a node u in OPEN minimizing g[u] + h[u]
    fmin <- g[frontierBest] + h[frontierBest]
    for each s in CLOSED:
        h[s] <- max(h[s], fmin - g[s])
    execute one or more actions on the path to frontierBest
```

The lookahead limit controls a computation–execution trade-off. More lookahead usually yields better actions but increases response time. Updating all expanded states makes the completed local search useful to later decisions.

3.6 Dynamic environments

When edges or costs change, restarting A* discards useful information. Incremental methods such as Lifelong Planning A* and D* Lite maintain local inconsistencies and repair only affected portions of the solution structure. The important distinction is:

- **adaptive heuristic search** changes estimates using experience;
- **incremental search** repairs a previous computation after problem changes.

Practical agents often do both.

3.7 From passive worlds to opponents

In the algorithms so far, successor costs are fixed by the environment. Now suppose another agent chooses actions deliberately against us. A frontier estimate is still needed, but a path is no longer determined only by our own choices. At alternating levels the decision rule changes from

$$\min_a \quad \text{or} \quad \max_a$$

to a nested alternation of our best choice and the opponent's worst reply. The next chapter inserts this alternation into the search tree.

> **Key idea.** Resource-bounded search and game search share the same architecture: expand a partial tree, evaluate its frontier, back information toward the root, and act. What changes is the backup operator.

3.8 Exercises

1. Show numerically how changing w changes node order on a graph of your choice.
2. Identify every assumption used by the Weighted A* bound. Give a failure mode when one assumption is removed.
3. Explain why an anytime algorithm should report both an incumbent and a bound.
4. Compare IDA* and A* in time and memory on a uniform tree.

5. Trace one RTAA* step with an expansion limit of three. Compute $f_{\min}$ and all updated heuristic values.
6. Give an example where a larger lookahead reduces planning episodes but increases wall-clock time before the first action.

Chapter 4

Adversarial Search and MinMax

Learning goals

- Formalize deterministic, perfect-information, two-player, zero-sum games.
- Distinguish a move sequence from a contingent strategy.
- Derive and trace the MinMax recursion.
- Explain optimal play and backward induction.
- Analyze time and memory in terms of branching factor and depth.

4.1 A formal game model

A deterministic two-player zero-sum game can be described by

$$\mathcal{G} = (S, s_0, P, A, T, Z, U),$$

where $P(s) \in \{\textsc{Max}, \textsc{Min}\}$ identifies the player to move, $A(s)$ is the legal action set, $T(s, a)$ is the successor, $Z \subseteq S$ is the terminal set, and $U : Z \to \mathbb{R}$ is utility for Max. Since the game is zero-sum, Min receives $-U$.

Typical utilities are $+1$ for a win, 0 for a draw, and -1 for a loss, but other scales are possible. Utilities encode preferences, not search costs.

4.2 Strategies, not paths

In single-agent search a solution is a path. In a game, the opponent decides which reply occurs, so a solution must prescribe an action after every relevant opponent response. Such a contingent plan is a *strategy*.

A pure strategy for Max maps each state at which Max moves to a legal action. MinMax asks for a strategy with the best guaranteed utility assuming that Min also acts optimally.

4.3 Deriving MinMax

At a terminal state, value is known. At a MAX state, MAX chooses the highest-valued successor. At a MIN state, MIN chooses the lowest-valued successor. Therefore

$$V(s) = \begin{cases} U(s), & s \in Z, \\ \max_{a \in A(s)} V(T(s,a)), & P(s) = \textsc{Max}, \\ \min_{a \in A(s)} V(T(s,a)), & P(s) = \textsc{Min}. \end{cases}$$

Listing 4.1: Recursive MinMax

```
function MINIMAX(s):
    if s is terminal:
        return U(s)
    if player(s) = MAX:
        value <- -infinity
        for each action a in actions(s):
            value <- max(value, MINIMAX(result(s,a)))
        return value
    else:
        value <- +infinity
        for each action a in actions(s):
            value <- min(value, MINIMAX(result(s,a)))
        return value
```

The root action is

$$a^* \in \arg\max_{a \in A(s_0)} V(T(s_0, a))$$

when MAX moves first.

4.4 Worked tree

Suppose the root is a MAX node with three MIN children. Their terminal children have utilities

$$(3, 5, 2), \qquad (9, 1, 4), \qquad (6, 7, 8).$$

The MIN backups are respectively $2, 1, 6$. The root selects

$$\max(2, 1, 6) = 6.$$

It is incorrect to choose the branch containing the leaf 9: MIN would respond with the leaf of value 1 in that branch.

4.5 Correctness by backward induction

Theorem 4.1 (MinMax optimality)**.** *In a finite deterministic perfect-information zero-sum game, the MinMax value is the utility that MAX can guarantee against an optimal opponent.*

Proof. Proceed by induction on the remaining depth. At depth zero the state is terminal and the claim follows from U. Assume every child value has the claimed meaning. At a MAX state, selecting a child of greatest value guarantees the maximum of those guarantees. At a MIN state, the opponent can select the least-valued child, so no larger guarantee is possible. The recursion therefore has the claimed meaning at the parent. □

This result depends on the model. Chance events require expectiminimax; hidden information requires beliefs or information sets; simultaneous moves may require mixed strategies. MinMax is not a universal model of all games.

4.6 Complexity

For branching factor b and terminal depth d, full MinMax visits

$$O(b^d)$$

nodes. A depth-first recursive implementation uses $O(bd)$ memory if it stores generated siblings, or $O(d)$ auxiliary stack memory with careful generation. The exponential time is decisive: chess-like games cannot be solved by simply searching the complete tree during play.

4.7 The relation to shortest-path search

Concept	Single-agent search	Adversarial search
Object returned	path	contingent strategy / root move
Frontier estimate	$h(s)$	$E(s)$
Backup	minimum path cost	alternating maximum and minimum
Accumulated quantity	$g(s)$	usually no path-cost sum
Terminal criterion	goal	win, loss, draw, or utility
Uncertainty	unexplored world	opponent's deliberate reply

Both algorithms expose a tree selectively and propagate frontier information backward. But $h(s)$ estimates remaining path cost, while a game evaluation estimates eventual utility under interaction. Treating them as numerically interchangeable leads to conceptual errors.

4.8 Exercises

1. Back up a depth-three tree containing alternating Max and Min nodes. Mark the selected root action and an optimal reply at every state.
2. Give a game tree in which choosing the highest visible leaf gives the wrong move.
3. Prove MinMax correctness using structural induction on a finite tree.
4. Explain why a MinMax solution is a strategy rather than one path.
5. Modify the recurrence for a chance node with known outcome probabilities.
6. Estimate the nodes in a game with $b = 30, d = 8$. Why is a tenfold processor speedup much less valuable than doubling effective search depth through pruning?

Chapter 5

Making MinMax Practical

Learning goals

- Use depth-limited MinMax and design a defensible evaluation function.
- Explain the horizon effect and terminal/evaluation scale compatibility.
- Derive alpha–beta pruning and state its correctness invariant.
- Relate move ordering to effective branching factor.
- Implement iterative deepening and a sound transposition table.

5.1 Depth-limited search

When the complete tree is too large, stop at depth d and replace unknown terminal utility with an evaluation $E(s)$:

$$V_d(s) = \begin{cases} U(s), & s \text{ terminal}, \\ E(s), & d = 0, \\ \max_a V_{d-1}(T(s,a)), & P(s) = \textsc{Max}, \\ \min_a V_{d-1}(T(s,a)), & P(s) = \textsc{Min}. \end{cases}$$

An evaluation function should correlate with eventual utility, be much cheaper than deeper search, respect the game's symmetries when appropriate, and put terminal outcomes outside the range of ordinary nonterminal estimates.

A linear hand-designed evaluation has the form

$$E(s) = \sum_{i=1}^{k} w_i \phi_i(s),$$

where features might encode material, mobility, threats, central control, or structure. Features should be measured from one consistent player's viewpoint. If the board is symmetric under reflection, unexplained asymmetric evaluation is a likely bug.

5.2 The horizon effect

A depth cutoff is a computational boundary, not a meaningful event in the game. A damaging move just beyond the boundary can be hidden by a temporary improvement just before it. This

is the horizon effect. Deeper search helps but does not remove the underlying discontinuity. Selective extension and quiescence search will be considered in chapter 6.

5.3 Alpha–beta bounds

Alpha–beta pruning computes the same root value as full MinMax while avoiding branches that cannot affect the decision.

Along the current path:

- α is the best value already guaranteed to MAX;
- β is the best value already guaranteed to MIN.

If $\alpha \geq \beta$, the remaining siblings cannot influence an ancestor's choice and may be pruned.

Listing 5.1: Depth-limited alpha–beta search

```
function ALPHABETA(s, depth, alpha, beta):
    if s is terminal:
        return U(s)
    if depth = 0:
        return E(s)

    if player(s) = MAX:
        value <- -infinity
        for each ordered action a in actions(s):
            value <- max(value,
                ALPHABETA(result(s,a), depth-1, alpha, beta))
            alpha <- max(alpha, value)
            if alpha >= beta:
                break
        return value
    else:
        value <- +infinity
        for each ordered action a in actions(s):
            value <- min(value,
                ALPHABETA(result(s,a), depth-1, alpha, beta))
            beta <- min(beta, value)
            if alpha >= beta:
                break
        return value
```

The root call uses $(-\infty, +\infty)$.

5.4 Why pruning is safe

Suppose a MIN node has already found a child of value $v \leq \alpha$. An ancestor MAX node already has an alternative worth at least α. Since MIN can force the current node to at most v, MAX will never choose this branch. Its remaining children cannot change the root decision. The dual argument applies at a MAX node when $v \geq \beta$.

Theorem 5.1 (Alpha–beta equivalence)**.** *For a fixed depth, evaluation function, terminal convention, and deterministic tie handling, alpha–beta search returns the same MinMax value as exhaustive depth-limited MinMax.*

Alpha–beta changes which parts of the tree become explicit, not the value defined by MinMax.

5.5 Move ordering

With poor ordering alpha–beta remains $O(b^d)$. With ideal ordering it examines roughly

$$O(b^{d/2}),$$

which is equivalent to roughly doubling the reachable depth for the same node budget. Useful ordering signals include:

- the best move from the previous iterative-deepening iteration;
- a transposition-table move;
- immediate wins, captures, or forcing moves;
- killer moves that previously caused a cutoff at the same depth;
- history scores accumulated across the search.

Ordering is a search heuristic: it need not be correct to preserve the final MinMax value, but a better ordering saves computation.

5.6 Iterative deepening

Iterative deepening searches depth $1, 2, \ldots, d$ in sequence. Although earlier nodes are revisited, the last level dominates an exponential tree. The method provides a legal move at interruption, improves move ordering, supplies transposition information, and makes time management predictable.

Listing 5.2: Iterative deepening driver

```
procedure ITERATIVE-DEEPENING(root, deadline):
    bestCompletedMove <- any legal move
    for depth <- 1 to infinity:
        result <- alpha-beta root search at depth with deadline checks
        if interrupted:
            return bestCompletedMove
        bestCompletedMove <- result.move
        use this iteration's principal variation to order the next one
```

Never replace the last completed result with a partially searched root unless the interruption protocol proves that doing so is safe.

5.7 Transposition tables

Different move orders can reach the same state. A transposition table stores results keyed by a hash of the complete position, including the side to move and rule-relevant history.

A useful entry stores:

$$(\text{key}, \text{depth}, \text{value}, \text{flag}, \text{bestMove}),$$

where the flag is EXACT, LOWER-BOUND, or UPPER-BOUND. A cutoff value is not necessarily exact. Reusing it as exact is a subtle and common error.

An entry is normally usable only if its stored search depth is at least the requested depth. Hash collisions must be detected with a sufficiently strong key or verification scheme. In games with repetition rules, the history required by the rules belongs to the state description.

5.8 Negamax as an implementation simplification

For symmetric zero-sum games one can evaluate from the current player's perspective and use

$$V(s) = \max_a[-V(T(s,a))].$$

Negamax shortens code but does not change the underlying theory. Sign errors are frequent: define clearly whose perspective E uses and test positions whose value is obvious.

5.9 Exercises

1. Run alpha–beta by hand on the tree in Chapter 4, first left-to-right and then in ideal order. Count evaluated leaves.
2. Prove that alpha–beta cannot prune a branch that would change the root value.
3. Design a five-feature evaluation for reduced Connect Four. State the sign, range, and justification of each feature.
4. Give a position pattern illustrating the horizon effect.
5. Explain why a transposition-table cutoff is sometimes only a bound.
6. Implement MinMax and alpha–beta and verify equality of returned values on thousands of randomly generated small trees.

Chapter 6

Adaptive Game-Playing Agents

Learning goals

- Integrate iterative deepening, ordering, caching, and selective extension.
- Explain quiescence search without confusing it with arbitrary extra depth.
- Identify what a game-playing agent can learn from previous searches.
- Generate and evaluate learning targets from search.
- Design a rigorous experiment comparing agent variants.

6.1 Where adaptation enters a game engine

A practical engine can improve at several levels:

1. **Value adaptation**: improve the frontier evaluation $E_\theta(s)$.
2. **Ordering adaptation**: predict moves likely to be strong or cause cutoffs.
3. **Allocation adaptation**: extend unstable or important branches selectively.
4. **Memory adaptation**: reuse searched states and recurring patterns.
5. **Time adaptation**: distribute a finite budget across moves and depths.

The final move is still justified by a search procedure; learning changes its estimates and computational priorities.

6.2 Quiescence search

At the nominal depth limit, do not immediately evaluate a tactically unstable position. Continue through a restricted set of forcing moves until a *quiescent* state is reached, then apply E.

Listing 6.1: Schematic alpha–beta quiescence search

```
function QUIESCENCE(s, alpha, beta):
    standPat <- E(s)
    if standPat >= beta:
        return beta
    alpha <- max(alpha, standPat)
```

```
    for each ordered forcing action a in forcingActions(s):
        score <- -QUIESCENCE(result(s,a), -beta, -alpha)
        if score >= beta:
            return beta
        alpha <- max(alpha, score)
    return alpha
```

The forcing set must be domain-specific and controlled. Extending every "interesting" move can cause search explosion or nontermination. Quiescence reduces tactical horizon errors; it does not solve strategic blindness.

6.3 Adaptive move ordering

Move-ordering statistics can be learned during search without changing MinMax semantics. A history heuristic increments a score when a quiet move produces a cutoff, often weighted by depth. Killer heuristics remember one or two moves that caused cutoffs at the same ply. Iterative deepening promotes the previous principal variation. These signals predict computational usefulness, not necessarily true game value.

This is analogous to adaptive single-agent search: the agent learns where to search, not only what answer to return.

6.4 Learning an evaluation function

For a linear evaluator

$$E_w(s) = w^\top \phi(s),$$

weights can be learned from labelled positions (s_i, y_i) by regression or classification. Targets may be terminal outcomes, expert assessments, or deeper-search values. A squared-error objective is

$$L(w) = \frac{1}{N}\sum_{i=1}^{N}(E_w(s_i) - y_i)^2 + \lambda\|w\|_2^2.$$

Nonlinear models E_θ can learn features as well as weights, but introduce larger data, calibration, and inference-cost requirements.

The evaluation scale should match the backup convention. If terminal wins are $+1$ and losses -1, a bounded prediction is convenient. If the engine uses large mate scores, training and search code must preserve their ordering and distance semantics.

6.5 Search as a teacher

A deeper search value can supervise a cheaper evaluator:

$$y(s) = V_D(s), \qquad E_\theta(s) \approx y(s),$$

where D is deeper than the depth at which the trained evaluator will usually be applied. This is a form of distillation. Self-play supplies positions without requiring a human label for each one.

There are risks:

- the student inherits systematic errors of the teacher search;
- self-play data are correlated and nonstationary;
- positions from current play may omit states reached by stronger opponents;
- a lower prediction error need not improve playing strength after search.

Therefore evaluate both prediction quality and head-to-head performance.

6.6 A modern search agent

A coherent deterministic game agent can use the following pipeline:

1. generate legal moves;
2. probe the transposition table;
3. order moves using prior search and learned or handcrafted signals;
4. run iterative-deepening alpha–beta;
5. extend unstable leaf positions with quiescence search;
6. evaluate stable leaves with E_θ;
7. store exact values or justified bounds;
8. update ordering statistics and log training examples;
9. return the best move from the last completed iteration.

Deep game systems may use different search algorithms, such as Monte Carlo Tree Search with learned policy and value networks. The transferable concept is the feedback loop between evaluation, selective tree construction, and experience.

6.7 Experimental methodology

To claim that adaptation helps, compare controlled variants:

- same legal-move generator and rules;
- same time or node budget;
- paired games with colours or first-player roles reversed;
- many starting positions, not only the initial board;
- fixed random seeds where randomness is used;
- confidence intervals for win rate or score;
- separate node count, wall time, depth, and playing result.

An ablation removes one component at a time: transposition table, learned ordering, quiescence, or learned evaluation. Otherwise interacting improvements cannot be attributed.

6.8 The completed conceptual arc

The six lectures can now be summarized as

$$\begin{gathered}\text{static estimate } h(s)\\ \downarrow\\ \text{estimate updated by experience}\\ \downarrow\\ \text{bounded partial search and action}\\ \downarrow\\ \text{alternating adversarial backup}\\ \downarrow\\ \text{pruning and efficient tree construction}\\ \downarrow\\ \text{learned evaluation and learned search control.}\end{gathered}$$

Key idea. A strong agent does not attempt to represent the entire future. It constructs the decision-relevant part of that future and improves the mechanisms that decide what is relevant.

6.9 Exercises

1. Define a quiescence condition for reduced Connect Four. Which forcing moves must be included, and how will termination be guaranteed?
2. Distinguish a move-ordering score from a position evaluation. Why can an incorrect ordering score improve speed without changing the result?
3. Propose a dataset and targets for learning a position evaluator from deeper search.
4. Design an ablation study for transposition tables, quiescence, and learned ordering.
5. Explain two forms of distribution shift in self-play training.
6. Write a one-page agent specification including state representation, search budget, evaluation, adaptation, and experimental protocol.

Appendix A

Solution Sketches

These sketches indicate the expected reasoning. They are not substitutes for a complete proof, trace, or implementation report.

Chapter 1

1. Use two routes to an intermediate state. Give the direct route a worse g but an artificially favourable successor heuristic, while keeping every heuristic below the true goal distance. The inconsistency permits the state to be closed before its best g is found.
2. $\max(h_1, h_2) \leq h^*$ because each argument is at most h^*. A sum may exceed h^* and is not generally admissible.
3. Greedy follows the smallest h; UCS the smallest g; A* the smallest sum. The cost-5 route is S, B, C, G.
4. Misplaced tiles counts at most one per required move. Manhattan distance sums individual displacement lower bounds and dominates misplaced tiles.
5. A goal can be generated by an expensive path while a cheaper route remains behind a frontier node of smaller f.

Chapter 2

1. At termination, expanded s has $g(s) + h(s) \leq g^*$; setting $h(s) = g^* - g(s)$ removes the observed underestimate relative to that episode.
2. Decreasing an edge cost can make an old value exceed the new true distance. Obstacle addition or nondecreasing costs preserve old lower bounds.
3. A correct trace must show monotone increases and distinguish movement cost from computation.
4. A local backup applies one Bellman operator at selected states; dynamic programming solves or iterates the coupled system broadly enough to converge.

Chapter 3

1. Compute both $g + wh$ keys. Larger w favours small h more strongly.

2. The standard bound needs an admissible heuristic, valid frontier lower bounds, appropriate duplicate handling/reopening, and goal acceptance on removal.
3. The incumbent says what can be executed; the bound says how far it may be from optimal.
4. IDA* uses much less memory but regenerates nodes across thresholds.

Chapter 4

1. Back up from leaves: maxima at MAX nodes and minima at MIN nodes.
2. Any branch with one spectacular leaf and one poor opponent reply suffices.
3. The base case is terminal utility; the induction step applies the optimal choice appropriate to the player.
4. One path assumes a particular reply. A strategy defines a response to all replies that can occur.
5. At a chance node use $V(s) = \sum_i p_i V(s_i)$.

Chapter 5

1. Good ordering reveals strong bounds early and therefore prunes more leaves.
2. At a cutoff, an ancestor already has an option that makes all possible refinements of the pruned branch irrelevant to its choice.
3. Features should include explicit perspective and must not double-count terminal outcomes.
4. A temporary gain at the last searched ply can hide a forced loss one ply later.
5. A cutoff proves only that a value is outside the active window; it may not determine the exact value.

Chapter 6

1. Include immediate wins and necessary blocks; ensure each recursive move reduces a finite tactical measure or impose a safe maximum extension.
2. Ordering changes discovery time of bounds; exhaustive alpha–beta semantics remain unchanged. Evaluation changes leaf values and can change the move.
3. Sample legal positions from diverse play, label them with completed deeper search values, and keep games or opening families separated across data splits.
4. Hold budget and all other components fixed, reverse first-player roles, and report paired outcomes plus search metrics and uncertainty.

Appendix B

Programming Project: A Search Agent in Two Worlds

B.1 Part A: Adaptive navigation

Implement a grid environment with blocked cells and positive movement costs. Provide UCS, A*, Weighted A*, Adaptive A*, and either LRTA* or RTAA*. The map should support a fixed goal and at least twenty start states. Add one family of maps containing heuristic depressions.

For every episode report:

- returned or executed path cost;
- number of expansions and generated nodes;
- peak frontier and stored-state counts;
- planning time and, for online search, travelled distance;
- heuristic change, for example number of updated states and total increase.

Visualize the map, path, expanded states, and learned heuristic as a scalar field. Explain at least one case in which the fastest method gives a worse path and one case in which adaptation transfers useful information across starts.

B.2 Part B: Reduced Connect Four

Implement legal play on a board small enough for controlled experiments but large enough that depth limits matter. Provide depth-limited MinMax, alpha–beta, iterative deepening, a transposition table, and a feature-based evaluation. Add at least one adaptive mechanism: history-based ordering, learned feature weights, or search-generated training targets.

Validate the program before comparing strength:

1. MinMax and alpha–beta must agree on small solvable positions.
2. Mirror-symmetric positions should receive mirror-consistent evaluations.
3. Table-enabled and table-disabled searches must return the same value.
4. Terminal wins must dominate all nonterminal evaluations.

Compare variants under equal node and time budgets. Report evaluated leaves, cutoffs, table hit rate, completed depth, move time, and paired match outcomes.

B.3 Suggested assessment rubric

Criterion	Weight
Correct state models and baseline algorithms	25%
Correct adaptive updates and alpha–beta/table semantics	25%
Experimental design and reproducibility	20%
Analysis connecting results to theory	20%
Code clarity, tests, and presentation	10%

Appendix C

Notation and Glossary

Term or symbol	Meaning
$g(n)$	Cost of the currently represented path from the search start to node n.
$h(n)$	Estimated cost from node n to a goal.
$h^*(n)$	True optimal cost from n to a goal.
$f(n)$	Search priority; in A*, $g(n) + h(n)$.
Admissible	Never overestimates the true remaining cost.
Consistent	Satisfies $h(s) \leq c(s, s') + h(s')$ for every transition.
Dominance	h_2 dominates h_1 when both are admissible and $h_2 \geq h_1$.
Heuristic depression	Region whose estimates are low relative to the cost of leaving it.
Online search	Alternation of planning, acting, observing, and updating.
Anytime algorithm	Returns a usable answer early and improves it with more computation.
$U(s)$	Exact utility of a terminal game state for MAX.
$E(s)$	Approximate evaluation of a nonterminal frontier state.
MinMax	Alternating max/min backup under optimal adversarial play.
α	Lower bound already guaranteed to MAX on the active path.
β	Upper bound already guaranteed to MIN on the active path.
Principal variation	Best move sequence found in the current completed search.
Transposition	The same game state reached through different move sequences.
Quiescence	Selective continuation from an unstable nominal leaf.
Effective branching factor	Empirical branching factor explaining an observed node count at a given depth.

Appendix D

Further Reading

Bibliography

[1] S. Russell and P. Norvig. *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2021.

[2] P. E. Hart, N. J. Nilsson, and B. Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 1968.

[3] R. E. Korf. Real-time heuristic search. *Artificial Intelligence*, 42(2–3), 1990.

[4] S. Koenig and M. Likhachev. Adaptive A*. In *Proceedings of AAMAS*, 2005.

[5] S. Koenig and X. Sun. Comparing real-time and incremental heuristic search for real-time situated agents. *Autonomous Agents and Multi-Agent Systems*, 18, 2009.

[6] J. Pearl. *Heuristics: Intelligent Search Strategies for Computer Problem Solving*. Addison-Wesley, 1984.

[7] D. E. Knuth and R. W. Moore. An analysis of alpha-beta pruning. *Artificial Intelligence*, 6(4), 1975.

[8] J. Schaeffer. The history heuristic and alpha-beta search enhancements in practice. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 11(11), 1989.

Closing Perspective

Heuristic search, real-time adaptation, and adversarial game playing are not isolated algorithmic tricks. They are instances of a general design problem: computation is limited, the future is only partially represented, and estimates must regulate which possibilities become explicit. A mature analysis therefore asks four questions together:

1. What value is the algorithm trying to compute?
2. What partial structure does it construct?
3. What estimate stands in for the unconstructed part?
4. How does experience improve the estimate or the construction policy?

Those questions provide a bridge from classical search to contemporary learned agents without hiding the algorithmic foundations.